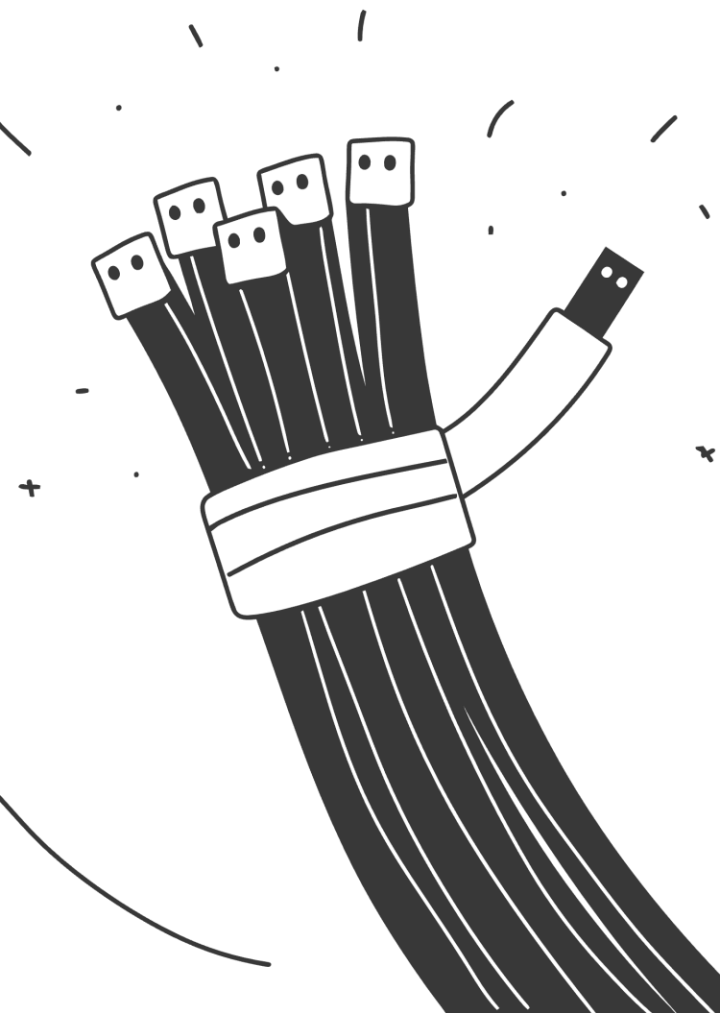


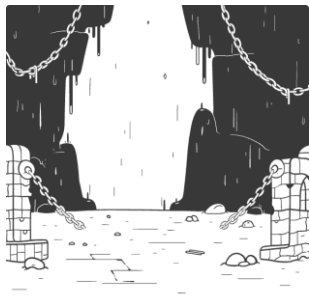


Hybrid Testing: API + UI

Session 6 — Writing resilient, isolated, full-stack tests with Playwright

FOR QA & AUTOMATION ENGINEERS

Why Shared Test Data Breaks Suites



TestMart starts with **12 seed products** — every test depends on them. If Test B deletes a product, Test A fails for the wrong reason.

❌ Tests that don't own their data are **fragile by design**.

The Fix: Each Test Owns Its Data

- **POST** → create exactly what the test needs
- Run the test
- **DELETE** → clean up afterwards

The request Fixture: API Without a Browser

```
test('GET /api/products returns catalogue',
  async ({ request }) => {
    const response = await request.get('/api/products');
    expect(response.status()).toBe(200);

    const body = await response.json();
    expect(body.count).toBe(12);
  });
```

Key Facts

- Full HTTP client **built into Playwright** — no browser launched
- Uses the same `baseUrl` from `playwright.config.ts`
- API calls take **milliseconds**; the same action via UI takes seconds

 The `--headed` flag has no effect — there is no browser to show.



Cross-Layer Validation: When UI and API Must Agree

UI-Only Test

Checks that *something* is displayed — not that it is *correct*.

Cross-Layer Test

Fetches truth from the API, then verifies the UI matches it exactly.

Bugs It Catches

Wrong product rendered, price formatting bug, stale cache showing old data.



`toHaveText(product.name)` is only meaningful because `product.name` came from the API — asserting **agreement between layers**, not just presence.

Login Bypass: Inject the Session Cookie

```
const { token } = await (await request.post(
  '/api/auth/login', {
    data: { email: 'standard_user@example.com',
            password: 'Password123!' }
  })).json();

await page.context().addCookies([{
  name: 'token', value: token,
  domain: 'localhost', path: '/'
}]);

await page.goto('/orders'); // no redirect
await expect(page.getByTestId('orders-heading'))
  .toBeVisible();
```

Why It Matters

Clicking the login form adds ~3 seconds per test. For **40 tests** that need auth, that is **2 minutes** of avoidable overhead.

Scales Further

`storageState` saves auth state once — zero login overhead across the entire suite.



Inject the cookie **before** `page.goto()` — not after.

API Preconditioning: Create → Verify → Clean Up



Create

Verify

Clean Up

Use `Date.now()` in the product name to guarantee uniqueness across parallel runs. Clean up in `afterEach` — always runs, even if the test fails. This makes each test **hermetic**: it neither depends on nor affects any other test.

Full-Stack Confidence: UI Confirms Render, API Confirms Storage

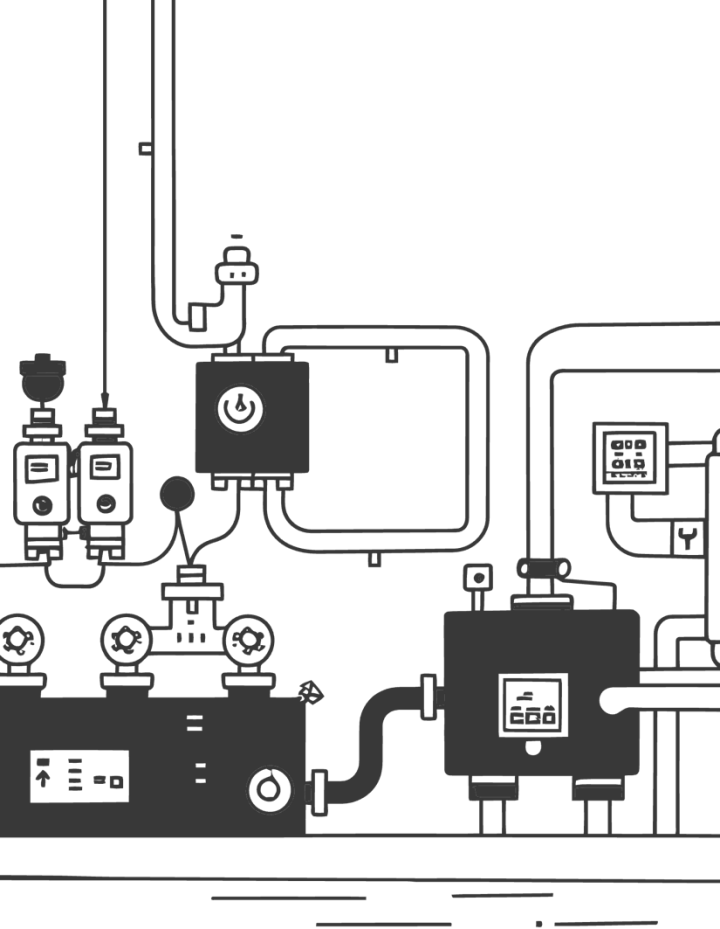
```
await page.getByTestId('place-order-btn').click();
await expect(page.getByTestId('order-confirmation'))
  .toBeVisible();
const orderId = (await page.getByTestId('order-id')
  .textContent())?.replace('#', '').trim();

const { order } = await (await request.get(
  `/api/orders/${orderId}`,
  { headers: { Cookie: `token=${token}` } }
)).json();
expect(order.status).toBe('confirmed');
expect(order.items).toHaveLength(1);
```

Two Assertions, Two Layers

- **UI assertion:** order confirmation page is visible
- **API assertion:** backend recorded the order with correct data

 A bug rendering the wrong order ID passes a UI-only test — but **fails the API check**. Neither layer alone is sufficient.



The `ApiClient` Class: POM for the API Layer

The Problem

Raw API calls scattered across tests = same problem as raw locators. Endpoints change, auth headers repeat, logic duplicates.

The Solution

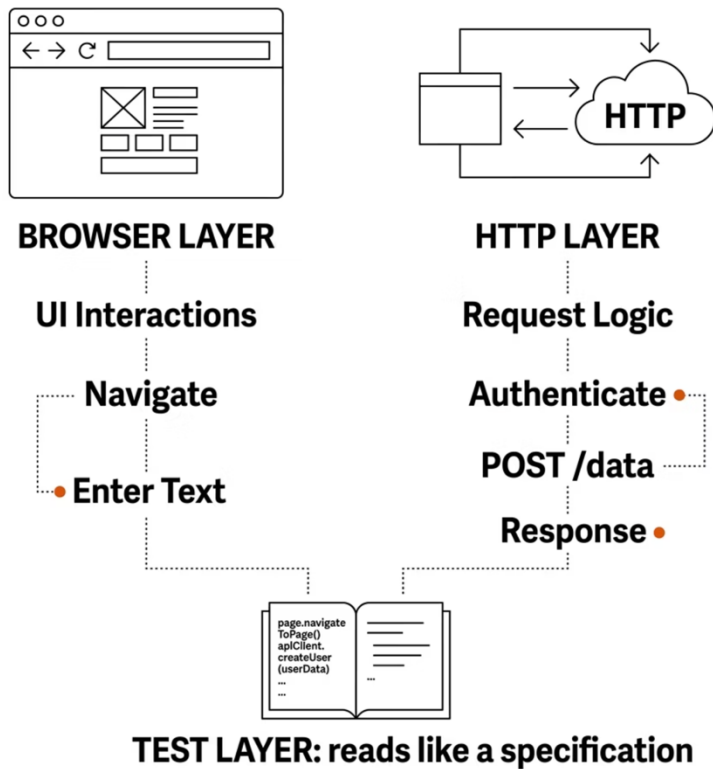
`ApiClient` encapsulates all API operations. Token stored on the instance: authenticate once, use everywhere.

In the Test

```
await api.loginAsAdmin();  
await api.createProduct({...});
```

No raw headers, no repeated auth logic.

POM vs. ApiClient: Same Principle, Different Layers



The Parallel

POM abstracts the **browser**. ApiClient abstracts **HTTP**. Together, your test reads like a specification at both layers.

Key Question

If the admin endpoint changes from `/api/auth/login` to `/api/v2/auth/login` — how many files need updating?

✓ With ApiClient: **one file**. Without it: every test that calls the endpoint.



Session — Key Takeaways

01

Own Your Data

Create and delete test data via API. Never rely on shared seed data in parallel suites.

02

Use the request Fixture

Playwright's built-in HTTP client is fast, config-aware, and works without a browser.

03

Validate Across Layers

UI confirms render; API confirms storage. Both checks are required for full confidence.

04

Encapsulate with ApiClient

Apply the Page Object principle to the API layer — one place to change, everywhere benefits.